

응답 프로토콜 상세

시뮬레이터가 클라이언트에 전송하는 모든 WebSocket 이벤트의 필드, 타입, 값 범위와 발생 조건을 정리한 인터페이스 명세서입니다.

WebSocket

Socket.IO

Realtime Events

KST (UTC+9)

DOCUMENT

Response Protocol

NAMESPACE

/simulator

EVENTS

9 types

목차

0. 공통 규칙

1. 접속 방법

- 1.1 엔드포인트 & 인증
- 1.2 Socket.IO 클라이언트 예시 (Node.js)
- 1.3 연결 성공/실패 흐름
- 1.4 연결 해제 및 재접속
- 1.5 연결 상태 확인 (ping/pong)

2. WebSocket 이벤트

- 2.1 userProfile
- 2.2 heartRate
- 2.3 stepCount
- 2.4 bloodPressure
- 2.5 weight
- 2.6 glucose
- 2.7 sleep
- 2.8 error
- 2.9 pong

본 문서는 시뮬레이터가 클라이언트에 실제로 내려주는 **모든 WebSocket 이벤트**의 필드, 타입, 값의 범위와 발생 조건을 상세히 정리합니다. 개략적인 스펙은 `docs/api.md`, 값 생성 규칙은 `docs/data-model.md`를 참고하고, 본 문서는 그 둘을 종합해 "실제 응답에 무엇이 어떻게 담기는지"에 집중합니다.

0. 공통 규칙

- 모든 WebSocket 이벤트 페이로드는 `{ "event": "<이벤트명>", "data": { ... } }` 형태로 감싸져 전송됩니다.
- `timestamp` 필드는 서버 실행 타임존과 무관하게 항상 **한국시간(KST, UTC+9)**로 계산되며, `+09:00` 오프셋이 붙은 ISO 8601 문자열입니다. (예: `2026-07-03T22:07:15.541+09:00`)
- 회원별 취침~기상 시간대("수면 중")는 회원 고유 기준 수면시간(`userId` 해시로 6.0~9.0시간 산출, KST 07시 기상 고정)을 기준으로 판정되며, 이 판정 결과가 심박수·걸음수·혈압·혈당 값 생성에 공통으로 반영됩니다.

1. 접속 방법

1.1 엔드포인트 & 인증

- 네임스페이스: `{ws|wss}://{server}/simulator`
- 인증은 별도 이벤트가 아니라 **연결(handshake) 시점의 쿼리 파라미터**로 이루어짐
- `userId`: 사용자 고유 ID
- `apiKey`: 해당 사용자의 API Key (`docs/data-model.md`의 `User.apiKey`와 일치해야 함)
- 예시

```
wss://healthsim.iranglab.com/simulator?userId=user_001&apiKey=key_001
ws://localhost:10000/simulator?userId=user_001&apiKey=key_001
```

1.2 Socket.IO 클라이언트 예시 (Node.js)

프로젝트 루트의 `test-socket.js`와 동일한 방식입니다.

```
const { io } = require('socket.io-client');

const socket = io('wss://healthsim.iranglab.com/simulator', {
  transports: ['websocket'], // polling 폴백 없이 websocket으로 고정
  query: { userId: 'user_001', apiKey: 'key_001' },
  reconnectionAttempts: 5,
  timeout: 5000,
});

socket.on('connect', () => console.log('connected', socket.id));
socket.on('userProfile', (msg) => console.log(msg));
socket.on('heartRate', (msg) => console.log(msg));
// stepCount, bloodPressure, weight, glucose, sleep 도 동일하게 socket.on(...)으로 구독
socket.on('error', (err) => console.error(err));
socket.on('disconnect', (reason) => console.log('disconnected:', reason));
```

- `transports: ['websocket']` 을 지정하지 않으면 Socket.IO가 기본적으로 HTTP 롱폴링으로 먼저 시도한 뒤 업그레이드하므로, 초기 이벤트 수신에 지연될 수 있어 명시적으로 고정하는 것을 권장합니다.

1.3 연결 성공/실패 흐름

1. 클라이언트가 위 URL로 연결하면 서버(`SimulatorGateway.handleConnection`)가 `userId` / `apiKey` 를 검증합니다.
2. **실패 시:** `error` 이벤트(`{ code: "AUTH_FAILED", message: "Invalid userId or apiKey." }`)를 보낸 뒤 서버가 즉시 소켓 연결을 끊습니다(재시도해도 같은 자격증명이면 계속 실패).
3. **성공 시:** `userProfile` 이벤트를 1회 보낸 뒤, 곧바로 `weight` / `bloodPressure` / `glucose` / `sleep` 을 각 1회 즉시 전송하고, 이어서 `heartRate` (4초) / `stepCount` (4.5초) / `bloodPressure` (2시간) / `glucose` (1시간) / `weight` (아침·점심·저녁) / `sleep` (매일 기상 시각) 주기 전송이 시작됩니다. 자세한 이벤트별 스펙은 아래 2장 참고.

1.4 연결 해제 및 재접속

- 클라이언트가 연결을 끊거나 네트워크가 끊기면 서버(`handleDisconnect`)가 해당 연결의 모든 타이머를 정리합니다(`destroySession`).
- **세션 상태는 저장되지 않습니다.** 재접속하면 완전히 새 세션으로 시작되어 `stepCount` 는 0부터, `weight` / `sleep` 의 "오늘 이미 보냈는지" 여부도 초기화된 상태로 다시 계산됩니다.
- 같은 `userId` 로 동시에 여러 소켓을 연결하면 서버 내부적으로 연결(`client.id`)마다 **독립된 세션**을 생성합니다 — 즉 두 연결의 `stepCount` 누적치나 오늘 전송 여부가 서로 공유되지 않고 각자 따로 계산됩니다.

1.5 연결 상태 확인 (`ping` / `pong`)

- 클라이언트가 임의 데이터로 `ping` 을 보내면 서버가 동일한 데이터를 `pong` 으로 그대로 반환합니다. 왕복 지연 측정이나 연결 확인 용도로 사용할 수 있습니다.

```
socket.emit('ping', { ts: Date.now() });
socket.on('pong', (msg) => console.log('rtt check', msg));
```

2. WebSocket 이벤트

2.1 userProfile

- 발생 시점: 연결(`handleConnection`) 인증 성공 직후, 최초 1회
- 소스: `simulator.gateway.ts` → `SimulatorService.buildProfile()` → `UserService.buildUserProfile()`

필드	타입	설명
<code>userId</code>	string	사용자 ID
<code>name</code>	string	이름
<code>age</code>	number	나이
<code>gender</code>	"M" "F"	성별
<code>heightCm</code>	number	키(cm)
<code>weightKg</code>	number	최신 체중 기록(<code>UserBodyRecord</code>) 또는 <code>User.weightKg</code> , 없으면 0
<code>bmi</code>	number	최신 BMI 기록 또는 <code>User.bmi</code> , 없으면 0
<code>hypertension</code>	boolean	<code>UserDisease</code> 에 <code>HYP</code> 매핑 존재 여부
<code>diabetes</code>	boolean	<code>UserDisease</code> 에 <code>DIA</code> 매핑 존재 여부
<code>heartDisease</code>	"myocardial_infarction" "arrhythmia" null	<code>MI</code> > <code>ARR</code> 우선순위로 판정, 둘 다 없으면 <code>null</code>
<code>diseases</code>	Array<{diseaseCode, name, nameKr}>	보유한 모든 질환 매핑
<code>otherConditions</code>	string[]	<code>HYP</code> / <code>DIA</code> / <code>MI</code> / <code>ARR</code> / <code>none</code> 을 제외한 질환 코드 목록 (예: <code>AST</code> , <code>SLP</code> , <code>CHO</code> , <code>ATH</code> , <code>THY</code>)

```
{
  "event": "userProfile",
  "data": {
    "userId": "user_003",
    "name": "박지훈",
    "age": 45,
    "gender": "M",
    "heightCm": 172,
    "weightKg": 88,
    "bmi": 29.8,
    "hypertension": true,
    "diabetes": true,
    "heartDisease": null,
    "diseases": [
      { "diseaseCode": "HYP", "name": "Hypertension", "nameKr": "고혈압" },
      { "diseaseCode": "DIA", "name": "Diabetes", "nameKr": "당뇨병" }
    ],
    "otherConditions": []
  }
}
```

2.2 heartRate

- 발생 시점: 4초 주기 (정상), 그리고 심장질환(**MI**) 또는 고혈압(**HYP**) 보유자는 10분 주기로 이상 수치가 **동일한 이벤트**로 추가 전송됨 (별도 이벤트명 없음)
- 소스: `simulator.service.ts` `sendHeartRate()` / `sendAbnormalEvent()`

필드	타입	설명
timestamp	string	KST ISO 8601
userId	string	사용자 ID
heartRate	number	심박수 (bpm)
source	"simulation" "abnormal_event"	일반 생성값인지 이상 이벤트인지 구분
note	string?	<code>source</code> 가 <code>abnormal_event</code> 일 때만 존재, 현재 고정값 <code>"Possible tachycardia detected."</code>

값 생성 로직 - 기준: `baselineHeartRate` (회원별 고유값) - 변동폭: 평상시 $\pm 5\text{bpm}$, 수면 중 $\pm 2.5\text{bpm}$, 부정맥 (**ARR**) 보유 시 변동폭 추가(평상시 $\pm 4\text{bpm}$, 수면 중 $\pm 2\text{bpm}$) - 고혈압/당뇨 보유 시 $+2\text{bpm}$ 가산 - 수면 중: 기준치의 약 82%로 낮춤, 하한 42bpm (평상시 하한 50bpm) - 이상 이벤트(10분 주기, **MI** / **HYP** 보유자만): 기준치(수면 중이면 82% 적용) + 20bpm + (MI 보유 시 $+10\text{bpm}$ 추가) + 0~10 난수, 하한 100bpm

```
{ "event": "heartRate", "data": { "timestamp": "2026-07-03T22:07:15.541+09:00", "userId": "user_003", "heartRate": 100, "source": "abnormal_event", "note": "Possible tachycardia detected." }
```

```
{ "event": "heartRate", "data": { "timestamp": "2026-07-03T22:10:00.000+09:00", "userId": "user_009", "hea
```

2.3 `stepCount`

- 발생 시점: 4.5초 주기
- 소스: `simulator.service.ts` `sendStepCount()`

필드	타입	설명
<code>timestamp</code>	string	KST ISO 8601
<code>userId</code>	string	사용자 ID
<code>stepCount</code>	number	해당 일자 누적 걸음 수
<code>dailyReset</code>	boolean	이번 틱에서 날짜가 바뀌어 0으로 초기화되었는지 여부

값 생성 로직 - 평상시: `max(10, baselineStepRate * 20 + 0~20 난수)` 만큼 매 틱 누적 - 수면 중: 90% 확률로 증가 0, 10% 확률로 1~5보만 증가 (화장실 이동 등 모사) - 날짜 변경 판정은 KST(UTC+9) 기준으로 이루어짐 — 자정(KST 00:00)에 `stepCount` 가 0으로 초기화되고 그 틱에서 `dailyReset: true` 가 1회 표시됨

```
{ "event": "stepCount", "data": { "timestamp": "2026-07-03T22:07:15.541+09:00", "userId": "user_003", "ste
```

2.4 `bloodPressure`

- 발생 시점: 접속 즉시 1회, 이후 2시간 주기
- 소스: `simulator.service.ts` `sendBloodPressure()`

필드	타입	설명
<code>timestamp</code>	string	KST ISO 8601
<code>userId</code>	string	사용자 ID
<code>systolic</code>	number	수축기 혈압 (mmHg)
<code>diastolic</code>	number	이완기 혈압 (mmHg)
<code>source</code>	string	항상 <code>"simulation"</code>

값 생성 로직 - 기준: 고혈압(`HYP`) 보유 시 수축기 140 / 이완기 90, 미보유 시 120 / 80 - 수면 중: 야간 혈압 하강 (nocturnal dipping) 반영 — 기준치에 0.88 곱함, 변동폭도 절반 - 변동폭: 평상시 수축기 ± 6 , 이완기 ± 4 / 수면 중 수축기 ± 3 , 이완기 ± 2 - 하한: 수축기 90, 이완기 55

```
{ "event": "bloodPressure", "data": { "timestamp": "2026-07-03T22:07:15.541+09:00", "userId": "user_003",
```

2.5 weight

- 발생 시점: 접속 즉시 1회, 이후 매일 아침/점심/저녁(KST 08/12/18시)에 각 1회
- 소스: `simulator.service.ts` `sendWeight()` / `checkWeightSchedule()`

필드	타입	설명
<code>timestamp</code>	string	KST ISO 8601
<code>userId</code>	string	사용자 ID
<code>weightKg</code>	number	체중(kg) — <code>userProfile</code> 과 동일 값(세션 중 고정)
<code>bmi</code>	number	BMI — <code>userProfile</code> 과 동일 값
<code>skeletalMuscleMassKg</code>	number	골격근량(kg), 매 전송 시점마다 재계산
<code>bodyFatPercentage</code>	number	체지방률(%), 매 전송 시점마다 재계산
<code>source</code>	string	항상 <code>"simulation"</code>

값 생성 로직 - 체지방률: Deurenberg 공식 $1.2 \times \text{BMI} + 0.23 \times \text{나이} - 10.8 \times \text{성별계수} (\text{남 } 1 / \text{여 } 0) - 5.4 + \pm 1\% \text{p}$ 무작위 변동, 5~50% 범위로 clamp - 골격근량: $\text{체중} \times (1 - \text{체지방률} / 100)$ 으로 구한 체지방량의 42~46% - 하루 중 최대 3회(08/12/18시)만 전송되므로 수면 시간대와는 겹치지 않아 별도 보정 없음

```
{ "event": "weight", "data": { "timestamp": "2026-07-03T22:07:15.541+09:00", "userId": "user_003"
```

"weight"

2.6 glucose

- 발생 시점: 접속 즉시 1회, 이후 1시간 주기
- 소스: `simulator.service.ts` `sendGlucose()`

필드	타입	설명
<code>timestamp</code>	string	KST ISO 8601
<code>userId</code>	string	사용자 ID
<code>glucoseMgDl</code>	number	혈당(mg/dL)
<code>status</code>	"normal" "elevated" "high"	140 이상 high, 110~139 elevated, 그 미만 normal
<code>source</code>	string	항상 <code>"simulation"</code>

값 생성 로직 - 공복 기준치: 당뇨(`DIA`) 보유 시 130, 미보유 시 95 - 변동폭: 평상시 당뇨 ± 10 / 비당뇨 ± 5 , 수면 중에는 각각 ± 5 / ± 3 으로 축소(식사가 없어 안정적) - 식후 스파이크: 수면 중이 아니고 KST 08/12/18시 이후 2시간 이내일 때만

적용 — 비당뇨 +15~30, 당뇨 +40~70 - 새벽 현상(dawn phenomenon): 수면 중이며 당뇨 보유자이고 기상 1.5시간 이내로 남았을 때 +10~25 추가 상승 - 하한: 70

```
{ "event": "glucose", "data": { "timestamp": "2026-07-03T22:07:15.541+09:00", "userId": "user_003", "glucose": 100 }
```

2.7 sleep

- 발생 시점: 접속 즉시 1회(직전 수면 기록), 이후 매일 기상 시각(KST 07시)에 1회
- 소스: `simulator.service.ts` `sendSleep()` / `checkSleepSchedule()`

필드	타입	설명
<code>timestamp</code>	string	KST ISO 8601
<code>userId</code>	string	사용자 ID
<code>sleepHours</code>	number	수면 시간(시간 단위, 소수 첫째 자리)
<code>quality</code>	"good" "fair" "poor"	수면 품질
<code>bedTime</code>	string	취침 시각(KST)
<code>wakeTime</code>	string	기상 시각(KST), 매일 07:00 고정
<code>source</code>	string	항상 <code>"simulation"</code>

값 생성 로직 - 기준 수면시간: `userId` 를 MD5 해시해 6.0~9.0시간 사이 값 고정 부여 (회원마다 다름, DB 스키마 변경 없이 계산) - 나이 보정: 40세 이상 -0.3시간, 60세 이상 -0.6시간 - 매 전송 시 기준치에 ± 0.6 시간 무작위 변동 - 수면무호흡 (SLP) 보유 시 0.8~1.8시간 추가 차감, 60% 확률로 품질을 무조건 `poor` 로 강제 - 품질 분류(수면무호흡 강제 케이스 제외): 7.5시간 $\uparrow$ `good`, 6~7.5시간 `fair`, 6시간 $\downarrow$ `poor` - `bedTime` 은 `wakeTime` (당일 07:00 KST)에서 `sleepHours` 만큼 역산

```
{ "event": "sleep", "data": { "timestamp": "2026-07-03T07:00:00.000+09:00", "userId": "user_003", "sleepHours": 7.5, "quality": "good", "bedTime": "2026-07-03T00:00:00.000+09:00", "wakeTime": "2026-07-03T07:00:00.000+09:00", "source": "simulation" }
```

2.8 error

- 발생 시점: 연결 시 `userId` / `apiKey` 인증 실패
- 소스: `simulator.gateway.ts` `handleConnection()`

필드	타입	설명
<code>code</code>	string	고정값 <code>"AUTH_FAILED"</code>
<code>message</code>	string	고정 문구 <code>"Invalid userId or apiKey."</code>

```
{ "event": "error", "data": { "code": "AUTH_FAILED", "message": "Invalid userId or apiKey." } }
```

인증 실패 시 이 이벤트 전송 직후 서버가 소켓 연결을 강제 종료합니다.

2.9 pong

- 발생 시점: 클라이언트가 ping 이벤트를 보낼 때 그대로 echo
- 페이로드: 클라이언트가 보낸 data 를 그대로 반환 ({ event: 'pong', data })